A FOCUSED PATH FOR JAVA DEVELOPERS

Python + FastAPI for AI

Only what you need for AI work, with Java and Python side by side.

Contents

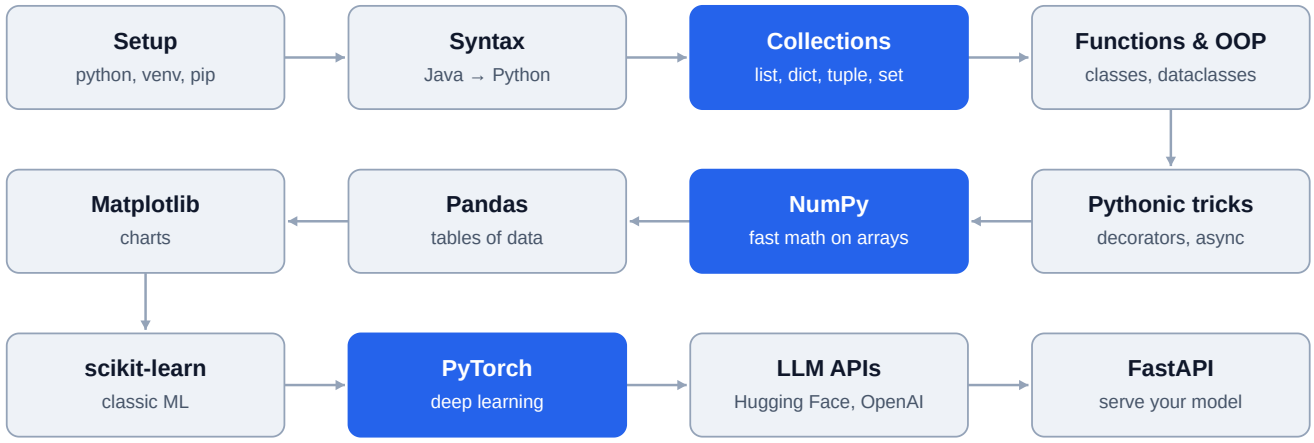
Roadmap	What you'll learn and in what order
1. Setup & tooling	Install Python, virtual environments, pip
2. Syntax for Java developers	Variables, conditions, loops, functions
3. Collections	list, dict, tuple, set and comprehensions
4. Classes & OOP	self, __init__, inheritance, dataclasses
5. Modules, errors, files	Imports, exceptions, with, *args / **kwargs
6. Decorators & async	The two features FastAPI is built on
7. NumPy	Arrays, shapes, vectorization, broadcasting
8. Pandas	DataFrames: load, filter, clean data
9. Matplotlib	Visualise data and training progress
10. scikit-learn	The ML workflow: fit, predict, evaluate
11. PyTorch	Tensors, neural networks, training loop
12. LLMs & Hugging Face	Use pre-trained models and LLM APIs
13. FastAPI	Spring Boot → FastAPI, serve a model
Study plan	Practice projects and resources

LESSON 1 OF 15

Roadmap

What you'll learn and in what order

You already know programming. You are not learning to code again — you are learning a **new accent**. Most concepts map directly from Java. This tutorial keeps only what you need for AI work and skips the rest.



The learning path. Highlighted steps are the core you will use every day in AI projects.

How to use this tutorial

- Go lesson by lesson. Each lesson shows Java on the left and Python on the right, so you can lean on what you know.
- Type every example yourself. Reading code is not the same as writing it.
- Finish the small exercise at the end of a lesson before moving on.

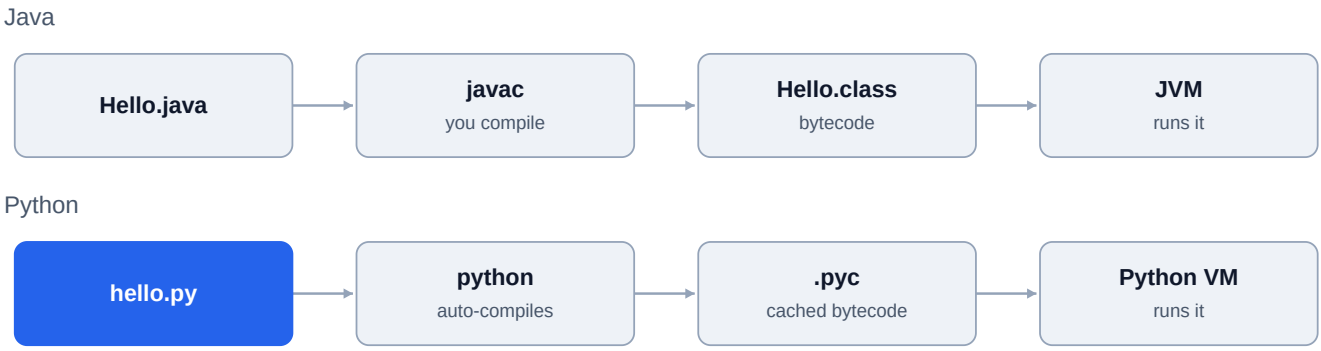
> **The one-sentence summary of Python vs Java**
>
> Python has no braces, no semicolons, no type declarations required, and no compile step — indentation is the syntax, and everything is an object.

LESSON 2 OF 15

1. Setup & tooling

Install Python, virtual environments, pip

How Python runs your code



Both languages compile to bytecode and run it on a virtual machine. Python just does it automatically when you run the file.

Step 1 — Install Python

- Download Python 3.12+ from python.org (on Windows, tick “Add Python to PATH”).
- Check it: `python --version` (on Mac/Linux it may be `python3`).
- Editor: VS Code or Cursor with the Python extension, or PyCharm (feels like IntelliJ).

Step 2 — Create a virtual environment (always, per project)

A virtual environment is a private folder of libraries for one project — like each Maven project having its own dependencies, instead of installing everything globally.

terminal

```
mkdir ai-learning && cd ai-learning
python -m venv .venv              # create the environment

source .venv/bin/activate         # Mac / Linux
.venv\Scripts\activate            # Windows

pip install numpy pandas matplotlib scikit-learn fastapi uvicorn
pip freeze > requirements.txt     # save versions (like pom.xml)
pip install -r requirements.txt   # restore on another machine
```

Java tooling → Python tooling

Java	Python	What it does
JDK	Python interpreter	Runs your code
Maven / Gradle	pip (or uv, poetry)	Installs libraries
pom.xml	requirements.txt / pyproject.toml	Lists dependencies
Maven Central	PyPI (pypi.org)	Library repository
Per-project dependencies	venv (.venv folder)	Isolated dependencies
JUnit	pytest	Testing
main() method	if __name__ == "__main__":	Entry point
jshell	python (REPL) / Jupyter Notebook	Try code interactively

> **Tip: use Jupyter for AI learning**
> Run `pip install jupyter` then `jupyter notebook`. A notebook lets you run code cell by cell and see charts inline — almost every AI course and Kaggle example uses it.

Your first program

Java

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, AI!");
    }
}
```

Python

```
# hello.py   → run with:  python hello.py
print("Hello, AI!")
```

LESSON 3 OF 15

2. Syntax for Java developers

Variables, conditions, loops, functions

Variables and types

No type declarations. The variable gets its type from the value. Types still exist and are strict — Python will not silently add a string to a number.

Java

```
int age = 30;
double price = 9.99;
String name = "Asha";
boolean active = true;
final int MAX = 100;
Object nothing = null;

String msg = "Hi " + name + ", age " + age;
```

Python

```
age = 30
price = 9.99
name = "Asha"
active = True          # capital T / F
MAX = 100              # constant by convention
only
nothing = None         # Python's null

msg = f"Hi {name}, age {age}"   # f-string
```

Java	Python	Note
int, long	int	Unlimited size — no overflow
float, double	float	64-bit like double
String	str	Single or double quotes, both fine
boolean	bool	True / False
null	None	Check with x is None
&& || !	and or not	Plain English words
a.equals(b)	a == b	== compares values in Python
a == b (same object)	a is b	Identity check
10 / 3 → 3	10 / 3 → 3.333, 10 // 3 → 3	/ is always real division
Math.pow(2, 8)	2 ** 8	Power operator

Indentation is the syntax

Braces are replaced by a colon `:` and 4-space indentation. Wrong indentation is a real error, not a style issue.

Java

```
if (score ⩾ 90) {
    grade = "A";
} else if (score ⩾ 75) {
    grade = "B";
} else {
    grade = "C";
}

String r = (x > 0) ? "pos" : "neg";
```

Python

```
if score ⩾ 90:
    grade = "A"
elif score ⩾ 75:
    grade = "B"
else:
    grade = "C"

r = "pos" if x > 0 else "neg"
```

Loops

Java

```
for (int i = 0; i < 5; i++) {
    System.out.println(i);
}

for (String n : names) {
    System.out.println(n);
}

while (count > 0) {
    count--;
}
```

Python

```
for i in range(5):          # 0,1,2,3,4
    print(i)

for n in names:
    print(n)

for i, n in enumerate(names):   # index + value
    print(i, n)

while count > 0:
    count -= 1               # no ++ or --
```

Functions

Java

```
public static int add(int a, int b) {
    return a + b;
}

// overloading for defaults
static String greet(String n) {
    return greet(n, "Hello");
}
static String greet(String n, String g) {
    return g + ", " + n;
}
```

Python

```
def add(a: int, b: int) → int:   # type hints
optional
    return a + b

def greet(name, greeting="Hello"):  # default
value
    return f"{greeting}, {name}"

greet("Ravi")                    # Hello, Ravi
greet("Ravi", greeting="Hi")     # keyword
argument

def min_max(nums):
    return min(nums), max(nums)  # return 2
values
lo, hi = min_max([3, 9, 1])
```

> **Common Java-habit mistakes**
> Forgetting the colon after `if` / `for` / `def`; writing `true` instead of `True`; writing `i++`; mixing tabs and spaces.

Exercise

Write `fizzbuzz(n)` that prints 1..n, but “Fizz” for multiples of 3, “Buzz” for 5, “FizzBuzz” for both.

LESSON 4 OF 15

3. Collections

list, dict, tuple, set and comprehensions

These four built-in types replace most of `java.util`. You will use `list` and `dict` constantly — JSON from any AI API is just nested lists and dicts.

Python	Java equivalent	Literal	Ordered?	Mutable?
list	ArrayList	[1, 2, 3]	Yes	Yes
tuple	immutable List / record	(1, 2, 3)	Yes	No
dict	HashMap (keeps insertion order)	{"a": 1}	Yes	Yes
set	HashSet	{1, 2, 3}	No	Yes

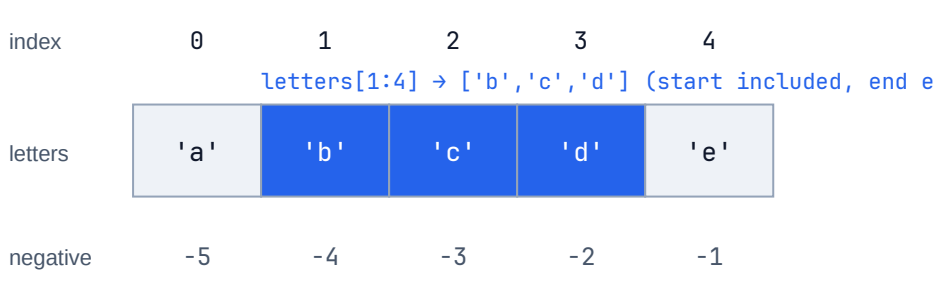
list — your ArrayList

Java

```
List<String> letters = new ArrayList<>(
    List.of("a","b","c","d","e"));
letters.add("f");
letters.get(0);
letters.get(letters.size() - 1);
letters.subList(1, 4);
letters.size();
letters.contains("c");
```

Python

```
letters = ["a", "b", "c", "d", "e"]
letters.append("f")
letters[0]          # 'a'
letters[-1]         # last item: 'f'
letters[1:4]        # slice: ['b','c','d']
len(letters)
"c" in letters      # True
```



Indexing and slicing. Negative indexes count from the end. A slice [start:end] includes start but excludes end — exactly like substring(1, 4).

dict — your HashMap

Java

```
Map<String,Integer> ages = new HashMap<>();
ages.put("Asha", 29);
ages.put("Ravi", 35);
ages.get("Asha");
ages.getOrDefault("Zoe", 0);
ages.containsKey("Ravi");
for (var e : ages.entrySet()) {
    System.out.println(e.getKey() +
e.getValue());
}
```

Python

```
ages = {"Asha": 29, "Ravi": 35}
ages["Meera"] = 41
ages["Asha"]            # 29  (KeyError if
missing)
ages.get("Zoe", 0)      # 0
"Ravi" in ages          # True
for name, age in ages.items():
    print(name, age)
```

tuple and set

Python

```
point = (3, 4)          # tuple: fixed, can't change
x, y = point            # unpacking
shape = (28, 28)        # you'll see tuples everywhere as array shapes

tags = {"ml", "ai", "ml"}   # set → {'ml', 'ai'}  duplicates removed
```

Comprehensions — Java Streams in one line

The single most “Pythonic” feature. Read it as: “give me X for each item in list, if condition”.

Java

```
List<Integer> squares = nums.stream()
    .filter(n → n % 2 == 0)
    .map(n → n * n)
    .collect(Collectors.toList());

Map<String,Integer> lens = words.stream()
    .collect(Collectors.toMap(w → w,
String::length));
```

Python

```
squares = [n * n for n in nums if n % 2 == 0]

lens = {w: len(w) for w in words}

total = sum(n * n for n in nums)   # generator,
no list built
```

Exercise

Given `words = ["apple", "banana", "kiwi", "cherry"]`, build a dict of only words longer than 4 letters mapped to their uppercase form, using one comprehension.

LESSON 5 OF 15

4. Classes & OOP

self, __init__, inheritance, dataclasses

Classes

`self` is Python's `this`, but you must write it explicitly as the first parameter. The constructor is always named `__init__`. There is no `new` keyword.

Java

```
public class Account {
    private String owner;
    private double balance;

    public Account(String owner, double
balance) {
        this.owner = owner;
        this.balance = balance;
    }

    public void deposit(double amt) {
        balance += amt;
    }

    @Override
    public String toString() {
        return owner + ": " + balance;
    }
}

Account a = new Account("Asha", 100);
```

Python

```
class Account:
    def __init__(self, owner: str, balance:
float = 0):
        self.owner = owner
        self._balance = balance   # _ =
"private" by convention

    def deposit(self, amt: float):
        self._balance += amt

    def __str__(self):            # like
toString()
        return f"{self.owner}: {self._balance}"


a = Account("Asha", 100)
print(a)       # Asha: 100
```

Inheritance

Java

```
public class Savings extends Account {
    private double rate;
    public Savings(String o, double b, double
r) {
        super(o, b);
        this.rate = r;
    }
}
```

Python

```
class Savings(Account):
    def __init__(self, owner, balance, rate):
        super().__init__(owner, balance)
        self.rate = rate
```

Dataclasses — Java records / Lombok

For simple data holders, a dataclass generates the constructor, equals and toString for you.

Java

```
public record Point(double x, double y) {}

var p = new Point(1.0, 2.0);
```

Python

```
from dataclasses import dataclass

@dataclass
class Point:
    x: float
    y: float

p = Point(1.0, 2.0)
print(p)    # Point(x=1.0, y=2.0)
```

“Dunder” (double underscore) methods

These let your objects work with Python syntax. PyTorch relies on them: calling `model(x)` actually runs `__call__`.

Method	Triggered by	Java analogy
__init__	Account(...)	constructor
__str__ / __repr__	print(obj)	toString()
__eq__	a == b	equals()
__len__	len(obj)	size()
__getitem__	obj[i]	get(i)
__call__	obj(x)	a functional interface's apply()

> **No interfaces? Duck typing**
> Python doesn't need `implements`. If an object has the method you call, it works (“if it walks like a duck…”). When you want a formal contract, use `abc.ABC` or `typing.Protocol`.

LESSON 6 OF 15

5. Modules, errors, files

Imports, exceptions, with, *args / **kwargs

Modules and imports

Every `.py` file is a module. A folder of modules is a package. No need for one-class-per-file.

project layout

```
my_project/
├── .venv/                  # virtual environment (don't commit)
├── requirements.txt
├── app/
│   ├── __init__.py         # marks the folder as a package
│   ├── main.py             # FastAPI app
│   └── model.py            # ML code
└── tests/
    └── test_model.py
```

Python

```
import numpy as np                  # import whole module with alias
from pathlib import Path            # import one thing
from app.model import predict       # your own package

if __name__ == "__main__":          # runs only when file is executed directly
    print(predict([1, 2, 3]))
```

Exceptions

Java

```
try {
    int x = Integer.parseInt(s);
} catch (NumberFormatException e) {
    System.out.println("bad: " +
e.getMessage());
} finally {
    cleanup();
}
throw new IllegalArgumentException("neg");
```

Python

```
try:
    x = int(s)
except ValueError as e:
    print(f"bad: {e}")
finally:
    cleanup()

raise ValueError("neg")   # no checked
exceptions
```

Files and the “with” statement

`with` is try-with-resources: the file closes automatically.

Java

```
try (var br =
Files.newBufferedReader(Path.of("data.txt"))) {
    String line;
    while ((line = br.readLine()) ≠ null) {
        System.out.println(line);
    }
}
```

Python

```
with open("data.txt") as f:
    for line in f:
        print(line.strip())

import json
with open("config.json") as f:
    config = json.load(f)       # → dict
```

Lambdas, *args and **kwargs

Python

```
people.sort(key=lambda p: p.age)       # like Comparator.comparing(...)

def log(*args, **kwargs):              # any positional / any named args
    print(args)     # tuple:  (1, 2)
    print(kwargs)   # dict:   {'level': 'info'}

log(1, 2, level="info")

params = {"lr": 0.01, "epochs": 10}
train(**params)                        # unpack dict into named args — very common in ML
```

LESSON 7 OF 15

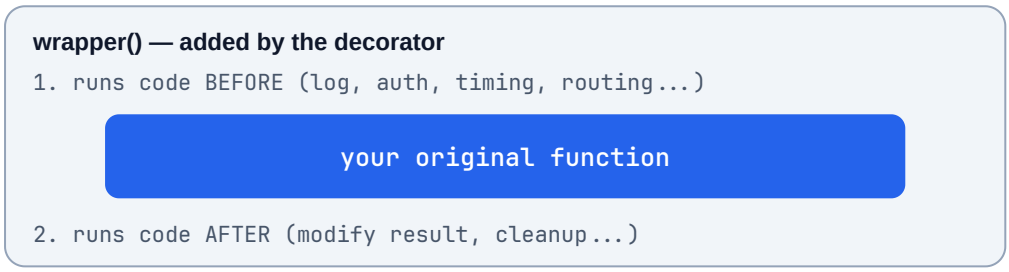
6. Decorators & async

The two features FastAPI is built on

Two features matter enormously for FastAPI and AI code: **decorators** and **async/await**. Learn them before FastAPI and it will feel easy.

Decorators — like Spring annotations, but real code

A decorator is a function that wraps another function. `@app.get("/")` in FastAPI looks exactly like Spring's `@GetMapping("/")`, but it is plain Python you could write yourself.



A decorator wraps your function with extra behaviour that runs before and after it.

Python

```
import time

def timed(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)        # call the original
        print(f"{func.__name__} took {time.time() - start:.2f}s")
        return result
    return wrapper

@timed                      # same as: train = timed(train)
def train():
    time.sleep(1)

train()                     # train took 1.00s
```

Generators — lazy streams

`yield` produces values one at a time, without loading everything into memory. Used for batching big datasets and for streaming LLM responses.

Python

```
def batches(items, size):
    for i in range(0, len(items), size):
        yield items[i:i + size]

for batch in batches(list(range(10)), 4):
    print(batch)     # [0,1,2,3]  [4,5,6,7]  [8,9]
```

async / await — like CompletableFuture, but readable

Use it when your code waits on I/O (HTTP calls to an LLM, a database). While one request waits, the server handles others.

Java

```
CompletableFuture<String> f =
    client.sendAsync(req, ofString())
          .thenApply(HttpResponse::body);
String body = f.join();
```

Python

```
import httpx, asyncio

async def fetch(url):
    async with httpx.AsyncClient() as client:
        r = await client.get(url)
        return r.text

body =
asyncio.run(fetch("https://example.com"))
```

Type hints — optional but recommended

Python

```
def top_k(scores: dict[str, float], k: int = 3) → list[str]:
    return sorted(scores, key=scores.get, reverse=True)[:k]

name: str | None = None      # Optional<String>
```

> **Why type hints matter for you**
> Python doesn't enforce them at runtime, but FastAPI and Pydantic read them to validate requests automatically and generate API docs. They are the bridge from your Java mindset.

LESSON 8 OF 15

7. NumPy

Arrays, shapes, vectorization, broadcasting

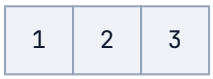
NumPy is the foundation of every AI library. Its core object, the `ndarray`, is a grid of numbers stored in one contiguous block of memory, with math implemented in C. Pandas, scikit-learn and PyTorch all build on these ideas.

terminal

```
pip install numpy
```

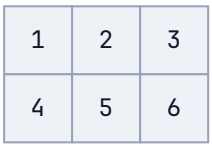
Shape: the most important concept

1-D (vector)



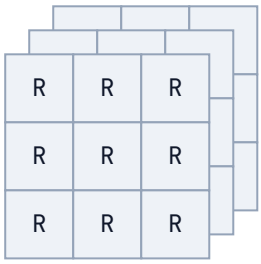
shape (3,)

2-D (matrix)



shape (2, 3)

3-D (e.g. RGB image)



shape (3, 3, 3)

Every array has a shape: a tuple of sizes per dimension. Most AI bugs are shape mismatches, so print .shape often.

Python

```
import numpy as np

a = np.array([1, 2, 3])
m = np.array([[1, 2, 3],
              [4, 5, 6]])

m.shape          # (2, 3)  → 2 rows, 3 columns
m.dtype          # int64
m[0, 1]          # 2        row 0, column 1
m[:, 0]          # [1, 4]   all rows, column 0
m.T              # transpose → shape (3, 2)
m.reshape(3, 2)  # same data, new shape

np.zeros((2, 3)); np.ones(5); np.arange(0, 10, 2); np.random.rand(3, 3)
```

Vectorization: no loops

Operations apply to every element at once. This is 10–100× faster than a Python loop.

Java

```
double[] out = new double[n];
for (int i = 0; i < n; i++) {
    out[i] = prices[i] * 1.18;
}
```

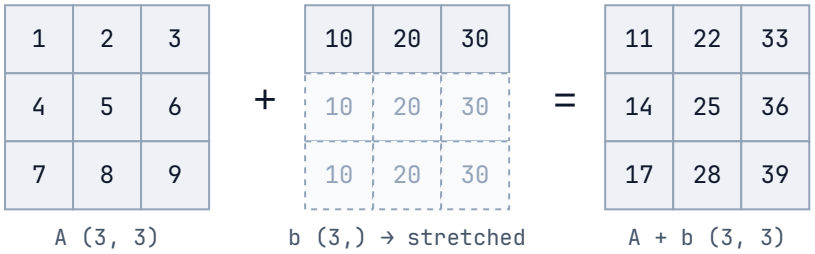
Python

```
out = prices * 1.18          # whole array at
once

mask = prices > 100          # [False, True,
...]
expensive = prices[mask]     # boolean
filtering

prices.mean(), prices.max(), prices.std()
```

Broadcasting



dashed rows are "virtual" copies — no memory used

Broadcasting: NumPy automatically stretches the smaller array to match the larger one, so a row can be added to every row of a matrix.

Matrix multiplication — the heart of neural networks

Python

```
X = np.random.rand(4, 3)     # 4 samples, 3 features
W = np.random.rand(3, 2)     # weights: 3 inputs → 2 outputs
Y = X @ W                    # shape (4, 2)   — one neural-network layer!
```

Shape rule for @

`(a, b) @ (b, c) → (a, c)`. The inner numbers must match.

LESSON 9 OF 15

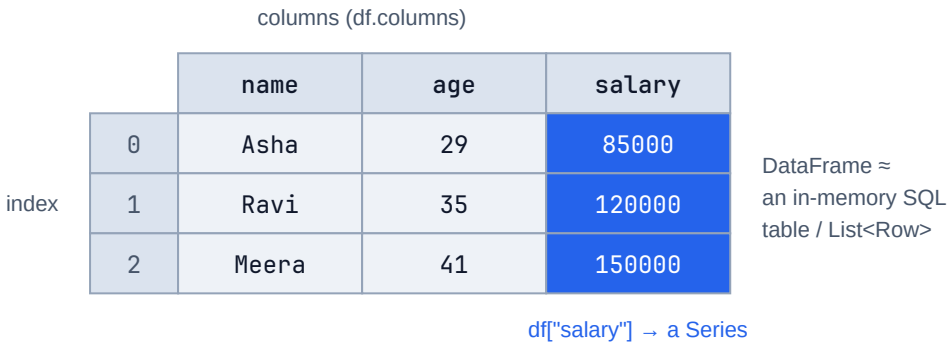
8. Pandas

DataFrames: load, filter, clean data

Pandas is for tabular data — CSVs, Excel, SQL results. Think of a `DataFrame` as an in-memory SQL table you can query with Python. Most of an ML project's time goes into this step.

terminal

```
pip install pandas
```



Anatomy of a DataFrame: labelled columns, a row index, and each column is a Series (a 1-D labelled array).

Python

```
import pandas as pd

df = pd.read_csv("employees.csv")    # also read_excel, read_json, read_sql

df.head()          # first 5 rows
df.shape           # (rows, columns)
df.info()          # column types, missing values
df.describe()      # count, mean, std, min, max per column
```

SQL → Pandas

SQL	Pandas
SELECT name, age FROM df	df[["name", "age"]]
WHERE age > 30	df[df["age"] > 30]
WHERE age > 30 AND dept = 'AI'	df[(df.age > 30) & (df.dept == "AI")]
ORDER BY salary DESC	df.sort_values("salary", ascending=False)
GROUP BY dept → AVG(salary)	df.groupby("dept")["salary"].mean()
JOIN other ON id	df.merge(other, on="id")
COUNT(*)	len(df)

Cleaning data for ML

Python

```
df.isna().sum()                          # missing values per column
df["age"] = df["age"].fillna(df["age"].median())
df = df.dropna(subset=["salary"])        # drop rows without a target
df = df.drop_duplicates()

df["salary_k"] = df["salary"] / 1000     # new column (vectorized)
df = pd.get_dummies(df, columns=["dept"])   # text categories → 0/1 columns

X = df.drop(columns=["salary"]).values  # features as NumPy array
y = df["salary"].values                 # target
```

> **Avoid loops over rows**
>
> If you find yourself writing `for i in range(len(df))`, there is almost always a vectorized way. Loops over rows are very slow in Pandas.

LESSON 10 OF 15

9. Matplotlib

Visualise data and training progress

You need charts to understand your data and to see if your model is learning. Matplotlib is the standard; Seaborn sits on top of it for nicer statistical plots.

terminal

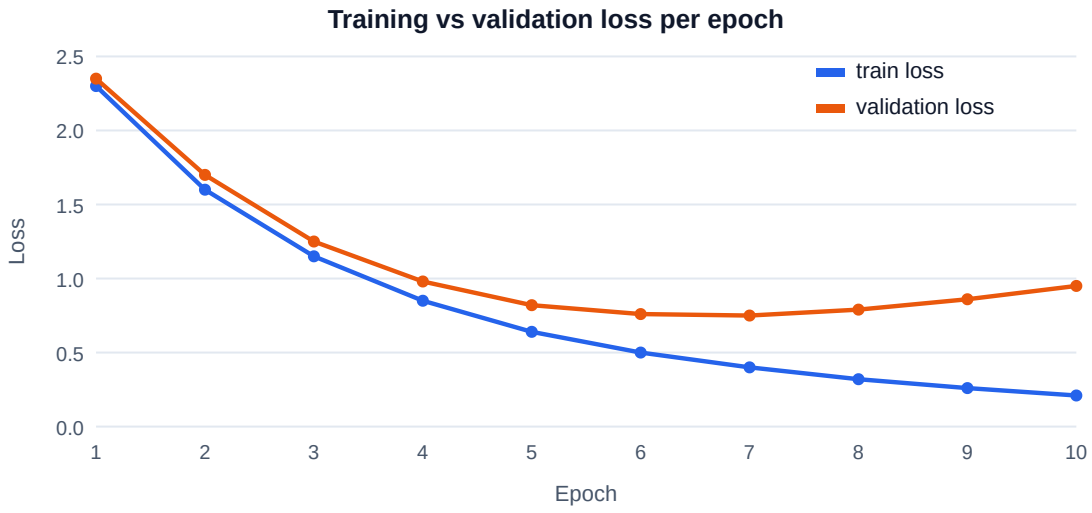
```
pip install matplotlib seaborn
```

Python

```
import matplotlib.pyplot as plt

plt.plot(epochs, train_loss, label="train loss")
plt.plot(epochs, val_loss, label="validation loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training progress")
plt.legend()
plt.show()

df["age"].hist(bins=20)              # distribution of a column
plt.scatter(df["age"], df["salary"]) # relationship between two columns
```



What the code above produces (illustrative numbers). When validation loss starts rising while training loss keeps falling, the model is overfitting — memorising instead of learning.

Chart	Use it to see	Code
Line	Change over time / epochs	plt.plot(x, y)
Histogram	Distribution of one column	plt.hist(values, bins=20)
Scatter	Relationship between two columns	plt.scatter(x, y)
Bar	Compare categories	plt.bar(names, counts)
Heatmap	Correlations / confusion matrix	sns.heatmap(df.corr(), annot=True)

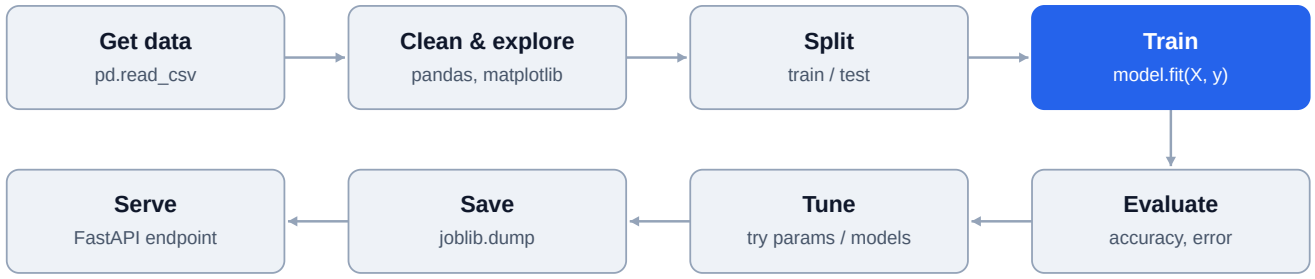
LESSON 11 OF 15

10. scikit-learn

The ML workflow: fit, predict, evaluate

scikit-learn is classic machine learning: regression, classification, clustering. Every model has the same API — `fit()` to learn, `predict()` to use. Learn it once, use 50 algorithms.

```
terminal
pip install scikit-learn
```



The machine learning workflow. The same steps apply to every project, from simple regression to deep learning.

Key vocabulary

Term	Meaning	Example
X (features)	Input columns — a 2-D array	age, experience, dept
y (target / label)	What you want to predict	salary or spam / not spam
Regression	Predict a number	House price
Classification	Predict a category	Is this email spam?
Training set	Data the model learns from (~80%)	
Test set	Unseen data to check it really works (~20%)	
Overfitting	Great on training data, poor on new data	Memorised the answers

A complete example — classify iris flowers

```python
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
import joblib

X, y = load_iris(return_X_y=True)            # 150 flowers, 4 measurements each

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

model = RandomForestClassifier(n_estimators=100)
model.fit(X_train, y_train)                  # learn

preds = model.predict(X_test)                # use
print("accuracy:", accuracy_score(y_test, preds))   # ~0.97

joblib.dump(model, "iris_model.joblib")      # save for the FastAPI lesson
```

> **Swap algorithms in one line**
>
> Replace `RandomForestClassifier` with `LogisticRegression`, `SVC` or `KNeighborsClassifier` — the rest of the code stays identical. Like coding to an interface in Java.

LESSON 12 OF 15

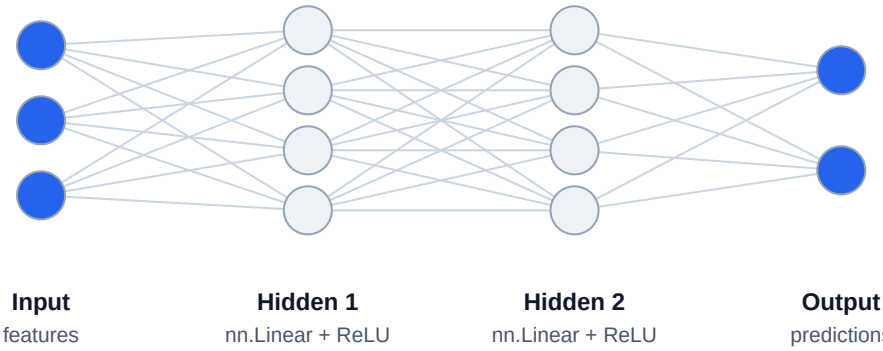
11. PyTorch

Tensors, neural networks, training loop

PyTorch is the main deep-learning library (used for LLMs, image models, and most research). A `Tensor` is a NumPy array that can run on a GPU and track gradients so the network can learn.

terminal

```
pip install torch      # see pytorch.org for the GPU-specific command
```



A neural network is layers of matrix multiplications (X @ W) with simple non-linear functions (like ReLU) between them.

Tensors

Python

```
import torch

t = torch.tensor([[1., 2.], [3., 4.]])
t.shape                    # torch.Size([2, 2])
t @ t                      # matrix multiply, same as NumPy
t.numpy()                  # → NumPy array
device = "cuda" if torch.cuda.is_available() else "cpu"
t = t.to(device)           # move to GPU if you have one
```

Define a model — it's just a class

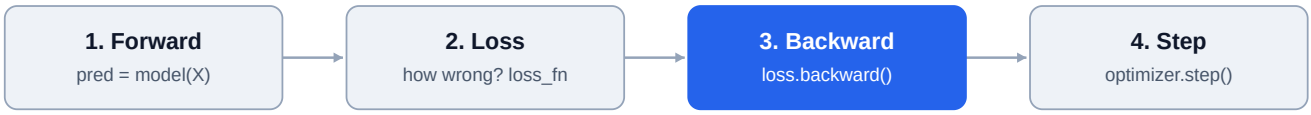
```python
import torch.nn as nn

class Net(nn.Module):                    # "extends nn.Module"
    def __init__(self):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Linear(4, 16), nn.ReLU(),
            nn.Linear(16, 16), nn.ReLU(),
            nn.Linear(16, 3),            # 3 output classes
        )

    def forward(self, x):                # what happens on model(x)
        return self.layers(x)

model = Net()
```

The training loop — memorise these 4 steps



Every PyTorch training loop repeats these steps for each batch, for several epochs (full passes over the data).

```python
X = torch.tensor(X_train, dtype=torch.float32)
y = torch.tensor(y_train, dtype=torch.long)

loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)

for epoch in range(100):
    pred = model(X)                  # 1. forward
    loss = loss_fn(pred, y)          # 2. loss
    optimizer.zero_grad()
    loss.backward()                  # 3. compute gradients
    optimizer.step()                 # 4. update weights
    if epoch % 10 == 0:
        print(epoch, loss.item())
```

> **You don't need to master the math first**
>
> Learn the loop, run it, change things, watch the loss chart. The intuition for gradients and backpropagation comes faster once you've seen it work.

LESSON 13 OF 15

12. LLMs & Hugging Face

Use pre-trained models and LLM APIs

Most AI work today uses pre-trained models instead of training from scratch. Two common ways: run open models locally with Hugging Face, or call a hosted LLM over HTTP.

Hugging Face Transformers — pre-trained models in 3 lines

terminal

```
pip install transformers torch
```

Python

```
from transformers import pipeline

sentiment = pipeline("sentiment-analysis")
print(sentiment("This tutorial is really easy to follow!"))
# [{'label': 'POSITIVE', 'score': 0.999}]

summarizer = pipeline("summarization")
generator = pipeline("text-generation", model="gpt2")
```

Calling a hosted LLM (OpenAI-style API)

terminal

```
pip install openai python-dotenv
```

Python

```
import os
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()                                    # reads OPENAI_API_KEY from a .env file
client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "You are a helpful tutor."},
        {"role": "user", "content": "Explain embeddings in one sentence."},
    ],
)
print(response.choices[0].message.content)
```

Notice that the request is just a list of dicts — the collections lesson pays off here. Model names and SDKs change often; check your provider's docs for the current ones.

Words you'll hear constantly

Term	Plain meaning
Token	A chunk of text (~¾ of a word) — what models read and bill by
Embedding	A list of numbers representing meaning; similar texts get similar numbers
Vector database	Stores embeddings and finds the most similar ones fast
RAG	Retrieve relevant documents, paste them into the prompt, then ask the LLM
Fine-tuning	Further training a pre-trained model on your own data
Inference	Using a trained model to make predictions

> **Never hard-code API keys**
> Keep them in a `.env` file (added to `.gitignore`) or environment variables.

LESSON 14 OF 15

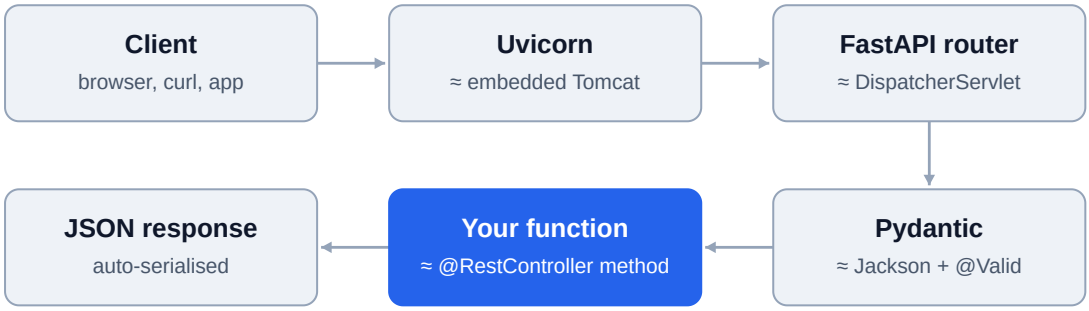
13. FastAPI

Spring Boot → FastAPI, serve a model

FastAPI is to Python what Spring Boot is to Java — but much smaller. It uses type hints to validate requests and auto-generates Swagger docs. It is the standard way to put an AI model behind a REST API.

terminal

```
pip install "fastapi[standard]"      # includes the uvicorn server
```



How a request flows through FastAPI, with the Spring Boot equivalent of each piece.

Hello API

Java

```
@RestController
public class HelloController {

    @GetMapping("/hello/{name}")
    public Map<String,String> hello(
            @PathVariable String name,
            @RequestParam(defaultValue="1") int
times) {
        return Map.of("msg", ("Hi " + name + "
").repeat(times));
    }
}
```

Python

```
# main.py
from fastapi import FastAPI

app = FastAPI()

@app.get("/hello/{name}")
def hello(name: str, times: int = 1):   # path
+ query param
    return {"msg": f"Hi {name} " * times}
```

terminal

```
fastapi dev main.py          # or:  uvicorn main:app --reload

# open http://127.0.0.1:8000/hello/Asha?times=2
# open http://127.0.0.1:8000/docs     ← free interactive Swagger UI
```

Validation for free

Call `/hello/Asha?times=abc` and FastAPI returns a clear 422 error, because the type hint says `int`. No extra code needed.

Request bodies with Pydantic — your DTOs

Java

```
public record Order(
    @NotBlank String item,
    @Min(1) int qty,
    String note) {}

@PostMapping("/orders")
public Order create(@Valid @RequestBody Order
o) {
    return o;
}
```

Python

```
from pydantic import BaseModel, Field

class Order(BaseModel):
    item: str
    qty: int = Field(ge=1)
    note: str | None = None

@app.post("/orders")
def create(order: Order) → Order:
    return order
```

Serve the ML model from the scikit-learn lesson

Python — main.py

```
from contextlib import asynccontextmanager
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import joblib

ml = {}

@asynccontextmanager
async def lifespan(app: FastAPI):
    ml["model"] = joblib.load("iris_model.joblib")   # load once at startup
    yield
    ml.clear()

app = FastAPI(title="Iris Classifier", lifespan=lifespan)

CLASSES = ["setosa", "versicolor", "virginica"]

class Flower(BaseModel):
    sepal_length: float
    sepal_width: float
    petal_length: float
    petal_width: float

class Prediction(BaseModel):
    species: str
    confidence: float

@app.post("/predict", response_model=Prediction)
def predict(f: Flower):
    features = [[f.sepal_length, f.sepal_width, f.petal_length, f.petal_width]]
    probs = ml["model"].predict_proba(features)[0]
    best = int(probs.argmax())
    if probs[best] < 0.5:
        raise HTTPException(status_code=422, detail="Not confident enough")
    return Prediction(species=CLASSES[best], confidence=float(probs[best]))
```

terminal

```
curl -X POST http://127.0.0.1:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"sepal_length":5.1,"sepal_width":3.5,"petal_length":1.4,"petal_width":0.2}'

# {"species":"setosa","confidence":1.0}
```

Async endpoint calling an LLM

Python

```
from openai import AsyncOpenAI
client = AsyncOpenAI()

class Question(BaseModel):
    text: str

@app.post("/ask")
async def ask(q: Question):
    r = await client.chat.completions.create(          # server stays free while waiting
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": q.text}],
    )
    return {"answer": r.choices[0].message.content}
```

Spring Boot → FastAPI cheat sheet

Spring Boot	FastAPI
@GetMapping / @PostMapping	@app.get / @app.post
@PathVariable	function param that appears in the path
@RequestParam	function param with a simple type
@RequestBody + DTO	function param typed as a Pydantic BaseModel
@Valid, @NotNull, @Min	Pydantic types and Field(ge=..., max_length=...)
ResponseStatusException	raise HTTPException(status_code=404)
@Autowired / DI	Depends(...)
@ControllerAdvice	@app.exception_handler(...)
Separate controllers	APIRouter + app.include_router(...)
springdoc / Swagger	Built-in at /docs
application.properties	pydantic-settings + .env file
MockMvc tests	fastapi.testclient.TestClient with pytest

LESSON 15 OF 15

Study plan

Practice projects and resources

A practical order of practice. Move on when you can do each checkpoint without looking at notes.

Stage	Focus	Checkpoint project
1	Setup, syntax, collections	Word-frequency counter: read a text file, print the top 10 words
2	Functions, OOP, modules, exceptions	A small bank-account module with tests in pytest
3	Decorators, generators, async	A @timed decorator and an async script fetching 5 URLs in parallel
4	NumPy + Pandas + Matplotlib	Analyse a Kaggle CSV (e.g. Titanic): clean, group, plot 3 charts
5	scikit-learn	Predict Titanic survival; reach > 75% test accuracy
6	FastAPI	Serve that model at POST /predict with Pydantic validation
7	PyTorch	Train a small network on MNIST digits; plot the loss curve
8	LLMs	A FastAPI /ask endpoint that answers questions about your own documents (simple RAG)

Recommended free resources (one per topic, no overload)

Topic	Resource
Python	The official Python Tutorial — docs.python.org/3/tutorial
NumPy	“NumPy: the absolute basics for beginners” — numpy.org
Pandas	“10 minutes to pandas” — pandas.pydata.org
scikit-learn	Getting Started guide — scikit-learn.org
PyTorch	“Learn the Basics” tutorial — pytorch.org/tutorials
FastAPI	Official Tutorial – User Guide — fastapi.tiangolo.com
Practice data	kaggle.com/datasets and Kaggle Learn micro-courses

The one habit that matters most

Build tiny projects, not more tutorials. After each lesson, close the document and rebuild the example from memory.